Prims Algorithm

AlgoExpert · Hard · Famous Algorithms

The problem

- Minimum spanning tree of a connected undirected weighted graph (adjacency list of [dest, weight]), grown from vertex 0 with a min-heap of frontier edges.

Intuition

- Grow one tree from a start vertex, always adding the cheapest edge leaving it.

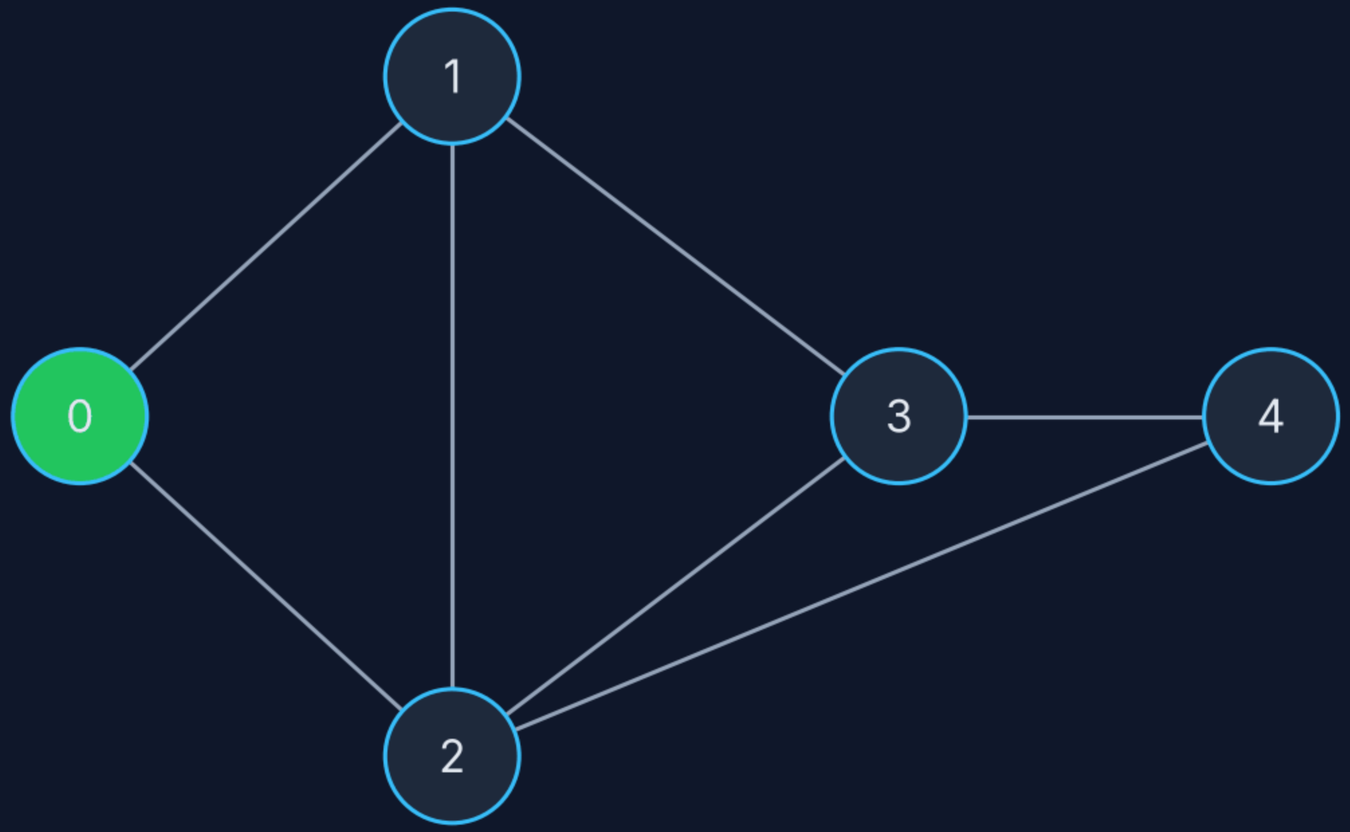
Brute force vs optimal

- Try all spanning trees → exponential
- Prim with array scan → $O(V^2)$
- Prim with a heap → $O(E \log V)$

Approach

- 1. Min-heap of edges from visited vertices
- 2. Pop the lightest to an unvisited vertex
- 3. Add it and push its edges
- 4. Repeat

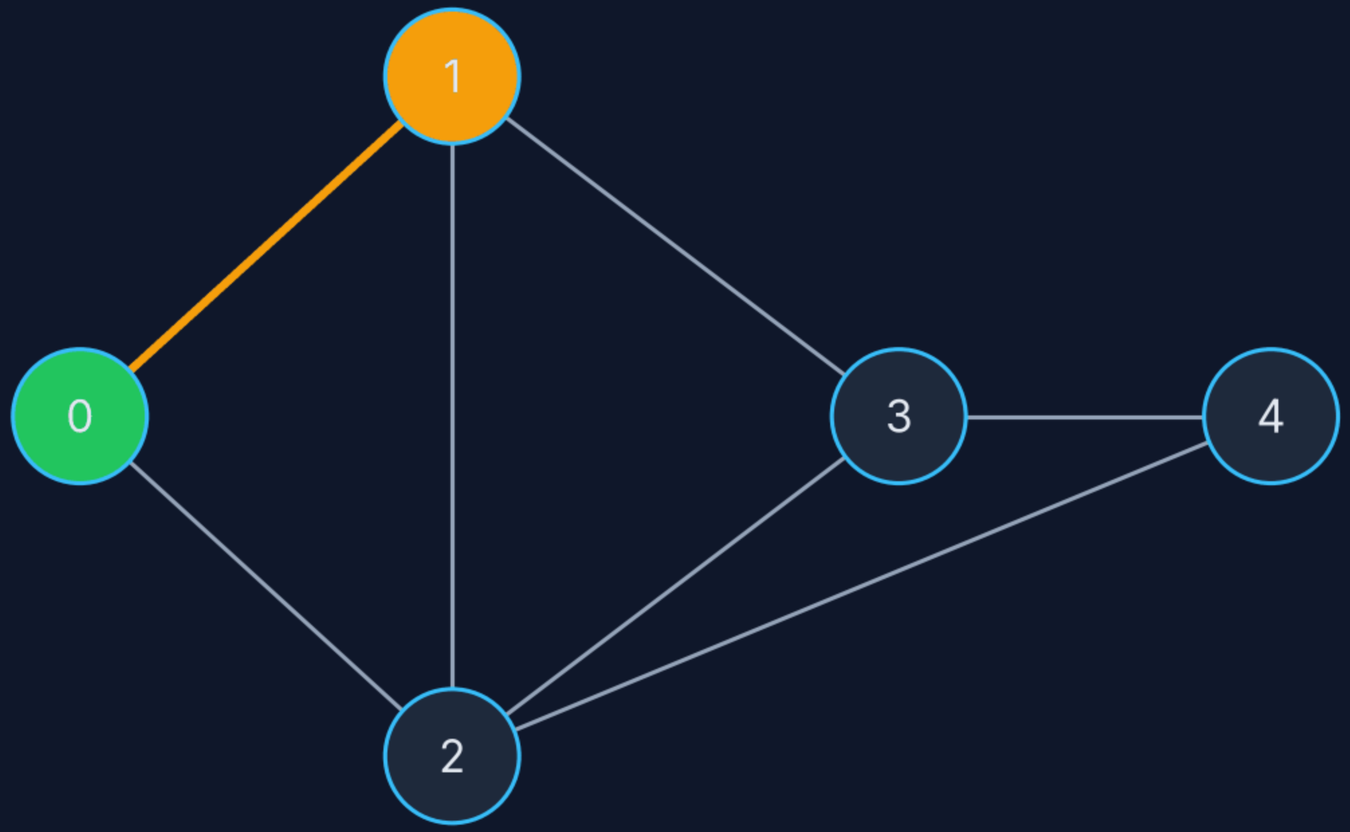
Worked example




```
tree = {0}
```

Start the tree at vertex 0

Worked example

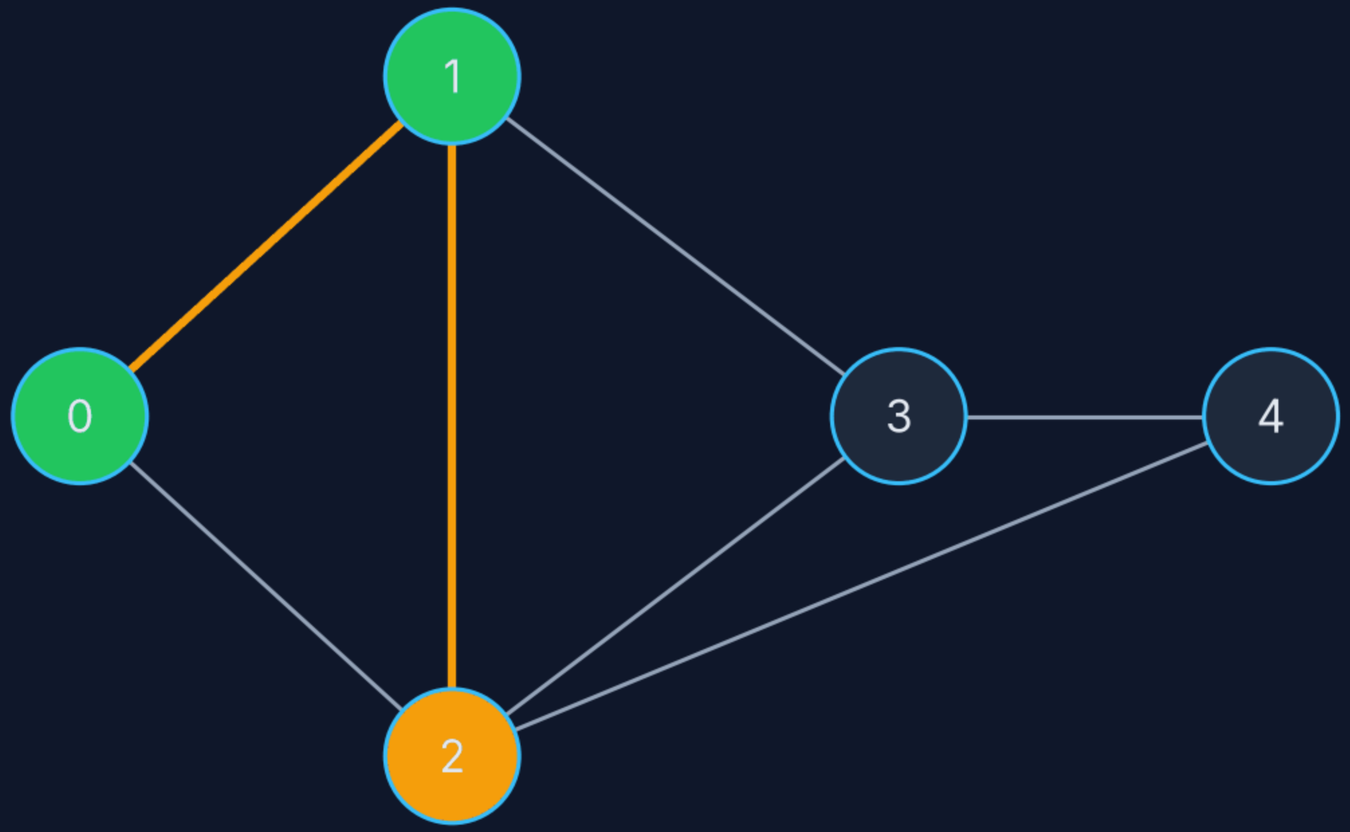



```
cheapest crossing edge 0–1 (3)

tree = [0, 1]
```

Add 0–1 (3)

Worked example

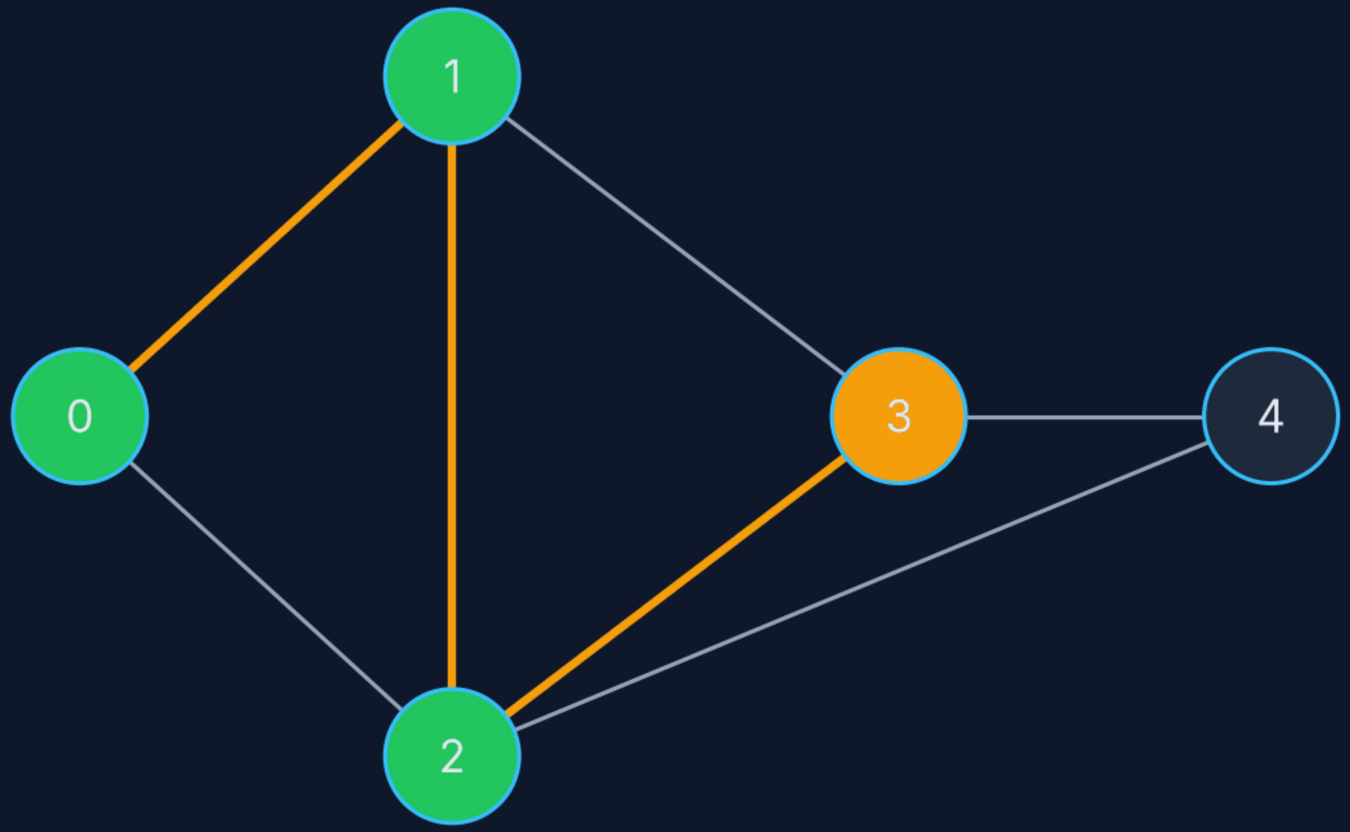



```
cheapest crossing edge 1–2 (1)

tree = [0, 1, 2]
```

Add 1–2 (1)

Worked example

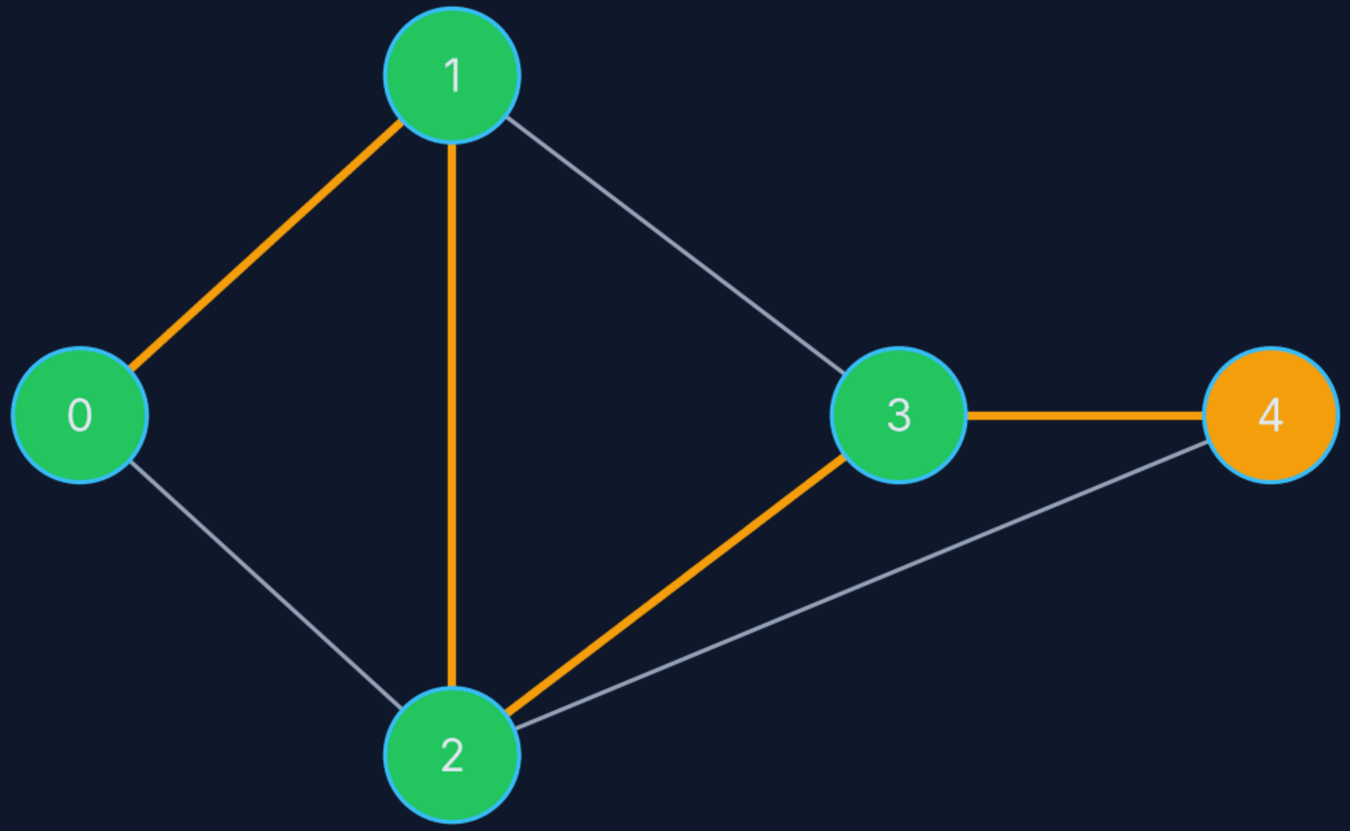



```
cheapest crossing edge 2–3 (2)

tree = [0, 1, 2, 3]
```

Add 2–3 (2)

Worked example




```
cheapest crossing edge 3–4 (6)

tree = [0, 1, 2, 3, 4]
```

Add 3–4 (6)

Java solution

```java
public class PrimsAlgorithm {
    public static int[][][] primsAlgorithm(int[][][] edges) {
        int n = edges.length;
        boolean[] in = new boolean[n];
        List<List<int[]>> mst = new ArrayList<>();
        for (int i = 0; i < n; i++) mst.add(new ArrayList<>());
        PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(e -> e[2]));
        in[0] = true;
        for (int[] e : edges[0]) pq.add(new int[]{0, e[0], e[1]});
        while (!pq.isEmpty()) {
            int[] e = pq.poll();
            if (in[e[1]]) continue;
            in[e[1]] = true;
            mst.get(e[0]).add(new int[]{e[1], e[2]});
            mst.get(e[1]).add(new int[]{e[0], e[2]});
            for (int[] f : edges[e[1]]) if (!in[f[0]]) pq.add(new int[]{e[1], f[0], f[1]});
        }
        int[][][] out = new int[n][][];
        for (int i = 0; i < n; i++) out[i] = mst.get(i).toArray(new int[0][]);
        return out;
    }
}
```

Complexity

- Time: O(e log v)
- Space: O(v + e)

Edge cases & pitfalls

- Skip edges into vertices already in the tree
- Disconnected graph
- Store edges both ways in the result

Interview tips & follow-ups

- Cut property
- Compare with Kruskal

Recap

- Grow one tree from a start vertex, always adding the cheapest edge leaving it.
- Complexity: O(e log v)